Exercises on Classes and Objects

Exercise 1: Good Wishes (መልካም ምኞት)

Instruction

Write a Java program that prints simple greeting messages in Amharic.

- Create a class named **MelkamMegnot** containing the **`main()`** method.
- Display greeting messages welcoming people.
- Print the greetings **"ሰላም ለሁላችሁም ይሁን!"** and **"እንኳን ደህና መጣችሁ!"** on the console.
- Run the program and observe the output.

Application

This type of program is useful for:

- Welcome screens in applications
- System startup messages
- User interface greetings in Ethiopian language applications

Solution

```
public class MelkamMegnot {
    public static void main(String[] args) {
        System.out.println("ሰላም ለሁላችሁም ይሁን!");
        System.out.println("እንኳን ደህና መጣችሁ!");
    }
}
```

Exercise 2: Ethipian Selamta (ሰላምታ)

Instruction

Write a Java program that demonstrates **user input and conditional statements** using Ethiopian greeting expressions.

- Create a class named **Selamta** with the **`main()`** method.
- Use the **Scanner** class to read input from the user.
- Ask the user to enter the time of day (**`qen`** or **`mata`**).
- Ask the user to enter the hour of the day.
- Use conditional statements (**`if-else`**) to determine the appropriate greeting message.
- Display the corresponding greeting in Amharic and English.

Applications

This logic can be used in:

- Voice assistants
- Customer service systems
- Smart home greeting systems
- Mobile applications that greet users based on time

Solution

```
import java.util.Scanner;
public class Selamta {
   public static void main(String[] args) {
       double hour;
       Scanner s = new Scanner(System.in);
       System.out.println("Enter \"qen\" or \"mata\": " );
       String timeOfDay = s.next().trim().toLowerCase();
       System.out.println("ስንት ሰዓት ነው?/What time is it?");
       hour = s.nextDouble();
       if(timeOfDay.equals("qen")) {
          if(hour >= 1 && hour < 6) {
             System.out.println("እንደምን አደሩ - Good morning");
          }else if(hour >= 6 && hour <= 12) {
             System.out.println("እንደምን ዋሉ - Good Afternoon.");
          }
       }else if(timeOfDay.equals("mata")) {
          if(hour >= 1 && hour < 4) {
             System.out.println("እንደምን አመሹ - Good Evening");
          }
          else {
             System.out.println("መልካም እረፍት - Good Rest");
          }
       }else {
          System.out.println("Invalid input");
       }
   }
}
```

Exercise 3: Coffee Ceremony (ቡና ማፍላት)

In the Ethiopian coffee ceremony there are items like **Jebena, Cups, Coffee beans**, etc.

Instruction

Write a Java program that demonstrates the concept of **Classes and Objects** using the Ethiopian coffee ceremony.

- Create a class called **CoffeeCeremony**.
- Include attributes such as **hostName** and **numberOfGuests**.
- Add methods that represent ceremony activities such as roasting beans, brewing coffee, and serving coffee.
- In another class containing the **`main()`** method, create an object of the **CoffeeCeremony** class.
- Initialize the object's attributes and call its methods to simulate the coffee ceremony.

Applications

This concept is useful in

- Event management systems
- Cultural heritage simulation software
- Hospitality management systems
- Educational cultural simulation application

Solution

```
class CoffeeCeremony {
    String hostName;
    int numberOfGuests;

    void roastBeans() {
        System.out.println(hostName + " is roasting coffee beans.");
    }
    void brewCoffee() {
        System.out.println("Coffee is brewing in the jebena.");
    }
    void serveCoffee() {
        System.out.println("Serving coffee to " + numberOfGuests + "
                           guests.");
    }
}
public class CoffeeDemo {
    public static void main(String[] args) {
        CoffeeCeremony ceremony1 = new CoffeeCeremony();
        ceremony1.hostName = "Amina";
        ceremony1.numberOfGuests = 5;
        ceremony1.roastBeans();
        ceremony1.brewCoffee();
        ceremony1.serveCoffee();
    }
}
```

Exercise 4: Market Day (ገበያ) → Message Passing

In OOP, **objects communicate by sending messages (method calls)**.
Market interaction example:

- Buyer → asks price
- Seller → responds

Instruction

Write a Java program to demonstrate **message passing between objects** using a market day scenario.

- Create a class named **Seller** with a method that sells an item and displays its price.
- Create another class named **Buyer** with a method that requests to buy an item from the seller.
- The buyer object should call the seller's method to complete the transaction.
- In the **`main()`** method, create objects of both classes and demonstrate how objects communicate through method calls.

Application

This model applies to:

- E-commerce platforms
- Online marketplaces
- Inventory and sales systems
- Client-server communication in distributed systems

Solution

```
class Seller {
    void sellItem(String item, int price) {
        System.out.println("Selling " + item + " for "+price+" birr.");
    }
}
class Buyer {
    void buyItem(Seller seller) {
        seller.sellItem("Tomatoes", 40);
    }
}
public class MarketDemo {
    public static void main(String[] args) {
        Seller seller = new Seller();
        Buyer buyer = new Buyer();
        buyer.buyItem(seller);
    }
}
```

Exercise 5: Exercise on Multiple Classes in the same file (Coffee Ceremony)

The **Ethiopian Coffee Ceremony (ቡና ሥነ-ሥርዓት)** is a structured cultural activity involving several objects and steps.

Instruction

Write a Java program demonstrating how **multiple classes can interact within the same program**.

- Create separate classes representing objects involved in the Ethiopian coffee ceremony such as **CoffeeBeans**, **Jebena**, and **Cups**.
- Each class should contain methods that perform their specific tasks (roasting, brewing, and serving).
- Create another class named **CoffeeCeremony** that uses objects of these classes.
- Include attributes such as **host** and **numberOfGuests**.
- Implement a method that performs the ceremony by calling the methods of the other classes.
- In the `main()` method, create a CoffeeCeremony object and execute the ceremony.

Application

This structure is useful for:

- Large software systems composed of multiple modules
- Restaurant management systems
- Workflow management systems
- Simulation software

Solution

```
class CoffeeBeans {
    void roast() {
        System.out.println("Roasting coffee beans on the pan...");
    }
}
class Jebena {
    void brew() {
        System.out.println("Brewing coffee in the jebena...");
    }
}
class Cups {
    void serve(int guests) {
        System.out.println("Serving coffee to " + guests + " guests.");
    }
}
class CoffeeCeremony {
    String host;
    int numberOfGuests;
    CoffeeBeans beans = new CoffeeBeans();
    Jebena jebena = new Jebena();
    Cups cups = new Cups();
    void performCeremony() {
        System.out.println("Host: " + host);
        System.out.print("Number of guests: ");
        System.out.println(numberOfGuests);
        beans.roast();
        jebena.brew();
```

```
        cups.serve(numberOfGuests);
    }
}
public class CoffeeDemo {
    public static void main(String[] args) {
        CoffeeCeremony ceremony = new CoffeeCeremony();
        ceremony.host = "Aster";
        ceremony.numberOfGuests = 5;
        ceremony.performCeremony();
    }
}
```

What is the expected output of the above program?

Exercise 6: Ethiopian Rivers.

Instruction

Write a Java program that illustrates **Classes and Objects** using Ethiopian rivers.

- Create a class named **River** representing the properties of a river.
- Include attributes such as **name**, **length**, **source**, and **destination**.
- Implement methods such as **flow()** and **displayInfo()**.
- In the **`main()`** method, create objects representing Ethiopian rivers such as **Abay River** and **Awash River**.
- Initialize their attributes and display their information.

Applications

This model can be used in:

- Geographic Information Systems (GIS)
- Environmental Monitoring software
- Educational geography applications
- Water resource management systems

Solution

```
class River {
    String name;
    double length;
    String source;
    String destination;
    void flow() {
       System.out.println(name + " is flowing.");
    }
    void displayInfo() {
       System.out.println("River Name: " + name);
       System.out.println("Length: " + length + " km");
       System.out.println("Source: " + source);
       System.out.println("Destination: " + destination);
       System.out.println();
    }
}
public class Main {
    public static void main(String[] args) {
          River abay = new River();
          abay.name = "Abay River";
          abay.length = 1450;
          abay.source = "Lake Tana";
          abay.destination = "Sudan";
          River awash = new River();
          awash.name = "Awash River";
          awash.length = 1200;
          awash.source = "Central Highlands of Ethiopia";
          awash.destination = "Lake Abbe";

          abay.displayInfo();
          abay.flow();

          awash.displayInfo();
          awash.flow();
    }
}
```

Exercise 7: Ethiopian Lakes

Instruction

Write a Java program that models **Ethiopian lakes** using classes and objects.

- Create a class named **Lake** with attributes such as **name**, **area**, and **region**.
- Implement a method to display lake information.
- In the **`main()`** method, create objects representing lakes such as **Lake Tana** and **Lake Ziway**.
- Initialize their attributes and display their details.

Applications

- Tourism Information systems
- Environmental research systems
- Geography learning software
- Lake resource management systems

Solution

```
class Lake {
    String name;
    double area;
    String region;

    void displayLake() {
       System.out.println("Lake: " + name);
       System.out.println("Area: " + area + " km²");
       System.out.println("Region: " + region);
       System.out.println();
    }
}
public class Main {
    public static void main(String[] args) {

       Lake tana = new Lake();
       tana.name = "Lake Tana";
       tana.area = 3050;
       tana.region = "Amhara";

       Lake ziway = new Lake();
       ziway.name = "Lake Ziway";
       ziway.area = 485;
       ziway.region = "Oromia";

       tana.displayLake();
       ziway.displayLake();
    }
}
```

Exercise 8: Heritage

Instruction

Write a Java program that demonstrates the use of **constructors in classes**.

- Create a class named **Heritage** with attributes **name** and **location**.
- Implement a **constructor** to initialize these attributes.
- Add a method to display the heritage site information.
- In the **`main()`** method, create objects representing Ethiopian heritage sites and display their details.

Applications

This structure is useful for:

- Tourism information systems
- Cultural heritage databases
- Museum management systems
- Educational cultural heritage applications

Solution

```
class Heritage {
    String name;
    String location;

    Heritage(String name, String location) {
       this.name = name;
       this.location = location;
    }
    void showInfo() {
       System.out.println(name + " is located in " + location + ".");
    }
}
public class HeritageDemo {
    public static void main(String[] args) {
       Heritage lalibela = new Heritage("Rock-Hewn Churches of
                                        Lalibela", "Amhara Region");
       Heritage axum = new Heritage("Obelisks of Axum", "Tigray
                                    Region");
       lalibela.showInfo();
       axum.showInfo();
    }
}
```

Exercise 9: Injera Making

Instruction

Write a Java program that models the **process of making injera** using a class and methods.

- Create a class named **Injera** with attributes such as ingredient, fermentation days, and cooking status.
- Use a **constructor** to initialize the ingredient and fermentation time.
- Implement methods representing steps in the cooking process such as mixing, fermenting, heating the mitad, and cooking.
- Include a method that checks whether the injera is ready.
- In the `main()` method, create an object of the Injera class and call its methods sequentially to simulate the cooking process.

Applications

This approach can be used in:

- Food production management systems
- Recipe simulation software
- Smart kitchen automation systems
- Culinary training applications

Solution

```
class Injera {
    private String ingredient;
    private int fermentationDays;
    private boolean mitadHeated;
    private boolean injeraDone;

    // Constructor
    public Injera(String ingredient, int fermentationDays) {
       this.ingredient = ingredient;
       this.fermentationDays = fermentationDays;
       this.mitadHeated = false;
       this.injeraDone = false;
    }
    // Methods representing steps
    public void mix() {
       System.out.println("Mix " + ingredient + " with water.");
    }
    public void ferment() {
       System.out.println("Let the mixture ferment for " +
                                 fermentationDays + " days.");
    }
    public void heatMitad() {
       mitadHeated = true;
       System.out.println("Heat the mitad (clay pan).");
    }
    public void cook() {
       if (mitadHeated) {
          System.out.println("Pour batter on the mitad, cover, and
                                        let it cook.");
          injeraDone = true;
       } else {
          System.out.println("Mitad is not heated yet!");
       }
    }
```

```
    public void checkStatus() {
       System.out.println("Injera ready: " + (injeraDone ? "Yes":
                                   "No"));
    }
}

public class InjeraDemo {
    public static void main(String[] args) {
       // Create an Injera object
       Injera injera = new Injera("Teff flour", 3);
       // Sequentially call methods
       injera.mix();
       injera.ferment();
       injera.heatMitad();
       injera.cook();
       injera.checkStatus();
    }
}
```